COMPLETE STUDY GUIDE · APRIL 2026

Hermes Agent

A self-improving AI agent with a closed learning loop — built for developers, businesses, and content creators alike.

PROJECT	BUILT BY	LICENSE
Hermes Agent v0.10.0	Nous Research	MIT Open Source

CONTENTS

1 What is Hermes Agent?
2 Core Concepts (Non-Technical)
3 How It Works — The Agent Loop
4 Architecture Deep Dive
5 The Tool Ecosystem
6 Memory & Learning System
7 Multi-Platform Gateway
8 Business Use Cases
9 Content Creator Use Cases
10 Developer Guide & Getting Started

SECTION 1

What is Hermes Agent?

> A plain-English overview of the project, its purpose, and why it matters — no technical background required.

The Big Picture

Hermes Agent is an **open-source AI assistant framework** built by Nous Research. Think of it as the engine behind a personal AI assistant that you own, control, and deploy anywhere — on your laptop, in a Docker container, on a remote server, or even as a serverless cloud function.

Unlike ChatGPT or Claude, which are services you access through a web browser, Hermes Agent is **software you run yourself**. You choose which AI model powers it (OpenAI, Anthropic Claude, Google Gemini, or any open-source model), where it runs, and what it can do.

💡 **The Key Analogy**

If ChatGPT is like renting an apartment (you use it but don't own it), Hermes Agent is like owning a house. You have full control: choose the layout, add rooms (tools/skills), and it gets smarter the longer you live there.

What Makes It Special?

It Learns From You

Unlike most AI tools that forget everything between conversations, Hermes builds a persistent memory of who you are, your preferences, and your past work sessions.

It Can Take Action

Not just a chatbot — Hermes can browse the web, write and run code, manage files, send messages, schedule tasks, and control your computer through 40+ built-in tools.

It Creates Its Own Skills

When Hermes solves a complex problem, it can save the solution as a reusable "skill" — so next time, it can do the same task faster and better without starting from scratch.

It Lives Everywhere

Chat with it via terminal, a web browser, Telegram, Discord, Slack, WhatsApp, or Signal. One agent, any interface — it works the same wherever you reach it.

The "Closed Learning Loop" Explained Simply

Most AI assistants work like this: you ask a question → they answer → they forget. Every conversation starts fresh. Hermes is different because it has a **closed learning loop**:

1. **You give Hermes a task**
   e.g., "Research the top 5 competitors in our market and write a report."
2. **Hermes completes the task using its tools**
   It searches the web, reads articles, synthesises the information, and writes the report.
3. **It saves what it learned as a "skill"**
   The exact steps it followed are saved so next time you ask for competitor research, it's instant.
4. **It updates your user profile**

It remembers you prefer bullet-point summaries with links, not long paragraphs — and applies this preference automatically.

5 **Next session, it's smarter**

Memory + skills from past sessions are loaded in, so it picks up exactly where you left off.

Who Is It For?

Developers

Build AI-powered workflows, automate DevOps tasks, do code reviews, and extend Hermes with custom tools and skills.

Businesses

Deploy private AI assistants that integrate with your existing tools (Slack, email, databases) without sending data to third-party services.

Content Creators

Automate content research, drafting, scheduling, and distribution across multiple platforms — all from a single AI assistant.

Open Source & Self-Hosted

The full source code is available on GitHub under the MIT license. You can inspect every line, modify it, and run it entirely on your own infrastructure — your data never leaves your machines unless you choose it to.

SECTION 2

Core Concepts (Non-Technical)

> Understanding the building blocks of Hermes without needing to know how to code.

The Five Pillars of Hermes

🤖 **The Agent — The Brain**
The core AI that understands your requests, plans what to do, uses tools to get things done, and crafts your final answer. Think of it as a very capable AI employee who can delegate tasks to specialists.

🔧 **Tools — The Hands**
Specific capabilities the agent can use: searching the web, running code, reading/writing files, sending messages, browsing websites, generating images, and more. The agent picks the right tools for each task automatically.

📖 **Skills — The Playbook**
Saved, reusable workflows for tasks you do repeatedly. Skills are like recipes — once written, you can run them with a single command (e.g., `/competitor-research`). Hermes ships with 27 categories of built-in skills.

🧩 **Memory — The Long-Term Knowledge Store**
Hermes can remember facts, preferences, past conversations, and project context across sessions. Unlike standard chatbots, it won't ask you the same questions twice.

🌉 **Gateway — The Delivery System**
The layer that lets you interact with Hermes from any messaging app (Telegram, Slack, Discord, WhatsApp, etc.) or interface (web, terminal, IDE). One AI brain, many access points.

How Conversations Work — Simply Put

💬 **Imagine you're texting a very capable assistant on Telegram:**

You send: *"Find me the latest papers on transformer efficiency and summarise them."*

> Behind the scenes, Hermes searches the web for papers, reads the key ones, compares findings, writes a clear summary, and sends it back. All in one message thread — no switching apps, no copy-pasting.

Providers vs. Models — What's the Difference?

Hermes is **model-agnostic**, meaning it doesn't care which AI model does the thinking — it can work with any of them:

PROVIDER	EXAMPLE MODELS	BEST FOR	COST
Anthropic	Claude Sonnet/Opus	Reasoning, writing, code	Paid API
OpenAI	GPT-4o, o3	General purpose	Paid API
Google	Gemini 2.0 Pro	Multimodal, large context	Paid API
Ollama / Local	Llama 3, Mistral, Qwen	Privacy, free, offline	Free
OpenRouter	Any of the above	Cost optimisation	Variable
AWS Bedrock	Claude on AWS	Enterprise compliance	Pay-per-use

You can switch models on the fly with a single command — your tools, skills, and memory all stay intact. Think of it like upgrading the "engine" in your car without needing a new car.

The Skills Library — Your Expanding Toolkit

Hermes ships with **27 categories of built-in skills** covering everything from GitHub workflows to creative content generation. You can also install community skills or write your own.

🔬 Research

Academic paper discovery, literature reviews, competitive intelligence

💻 Dev Tools

Code review, GitHub workflows, deployment pipelines

📱 Social Media

Post scheduling, content repurposing, analytics collection

🎨 Creative

Art generation prompts, music composition, storyboarding

📊 Data Science

Jupyter notebooks, data analysis, visualisation

🏠 Home Automation

Home Assistant integration, smart device control

✅ Productivity

Task management, meeting notes, email drafting

🔐 Security

Security audits, penetration testing (opt-in)

🍎 Apple / macOS

iMessage integration, macOS automation

SECTION 3

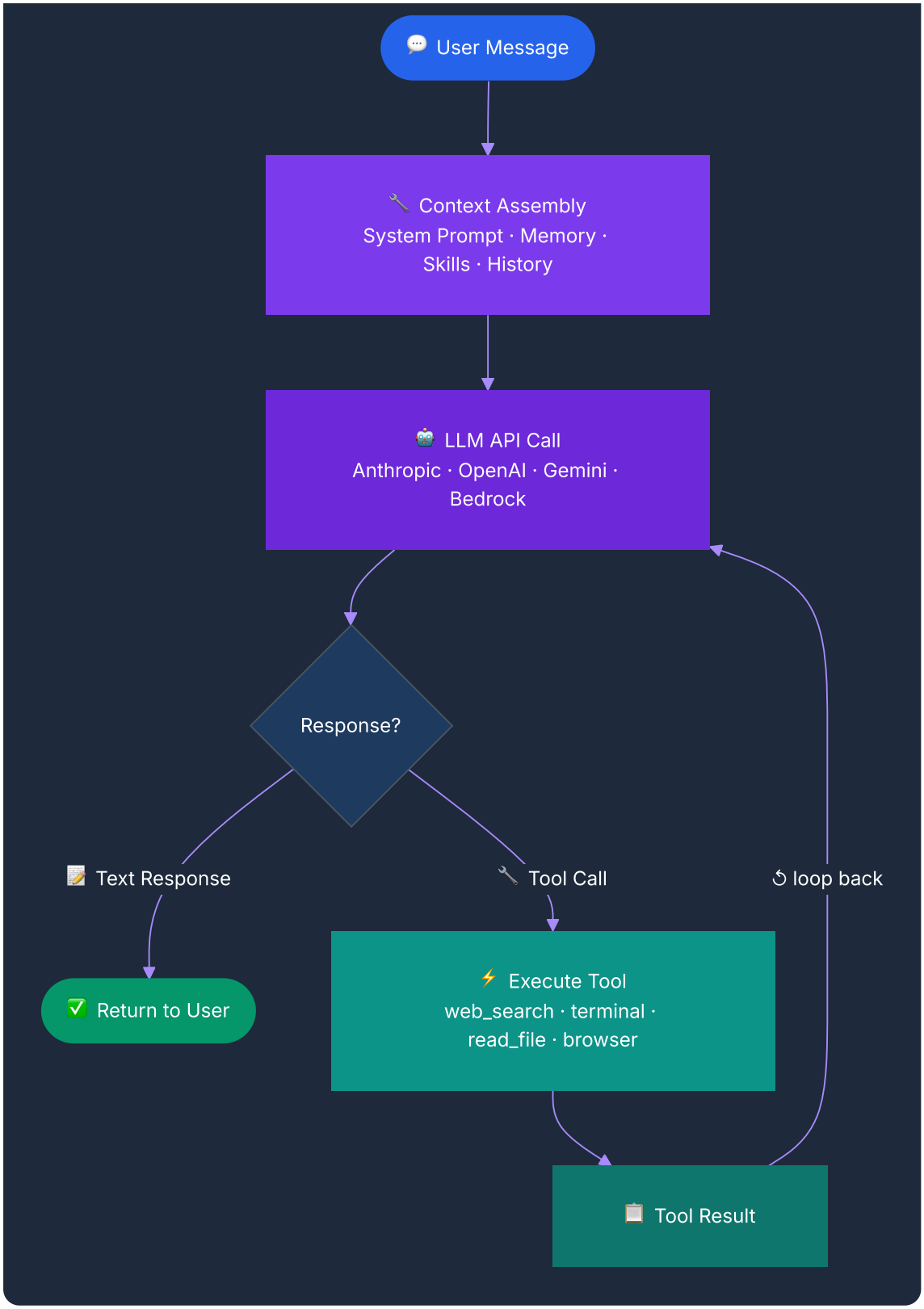
How It Works — The Agent Loop

> The technical heartbeat of Hermes: the conversation and tool-calling loop that drives everything.

The Core Loop (Technical)

Every interaction with Hermes runs through the **AIAgent class** defined in `run_agent.py`. This is the engine — roughly 12,000 lines of Python that orchestrate context, tool calls, provider communication, and response streaming.

THE HERMES CONVERSATION LOOP



Step-by-Step Breakdown

1

Context Assembly

Before sending anything to the AI model, Hermes builds a "context package" consisting of: a dynamic system prompt, relevant memories from past sessions, any active skills, the

current conversation history, and optional context files (documents you've loaded). Token budgeting ensures this never exceeds the model's context window.

2

LLM API Call

The assembled context is sent to whatever AI model is configured. The model receives a list of "tool schemas" — descriptions of every available tool including what it does and what parameters it accepts. The model can then decide to either reply directly or call one or more tools.

3

Tool Execution (if needed)

If the model decides to use a tool, Hermes parses the tool call, validates the parameters, executes the tool (e.g., runs a web search, executes code, reads a file), and appends the result back to the conversation. The model then sees the result and decides: reply now, or call another tool?

4

Loop Until Done

The tool-calling loop repeats until the model produces a text-only response (no more tool calls). There's a configurable maximum iteration limit (default: 90 turns) to prevent runaway loops.

5

Response + Memory Update

The final text response is streamed back to the user. Optionally, the session is saved to SQLite with full-text search indexing so past conversations can be recalled in future sessions.

The Context Budget System

Every AI model has a "context window" — a maximum amount of text it can process at once. Hermes manages this carefully through the **context compressor** (`agent/context_compressor.py`):

⚖️

How Token Budgeting Works

When the conversation approaches the model's context limit, Hermes automatically summarises older messages, keeping the most recent context intact. This allows conversations to run indefinitely without losing coherence. The user never has to manually manage this — it happens silently in the background.

Extended Thinking (Reasoning Mode)

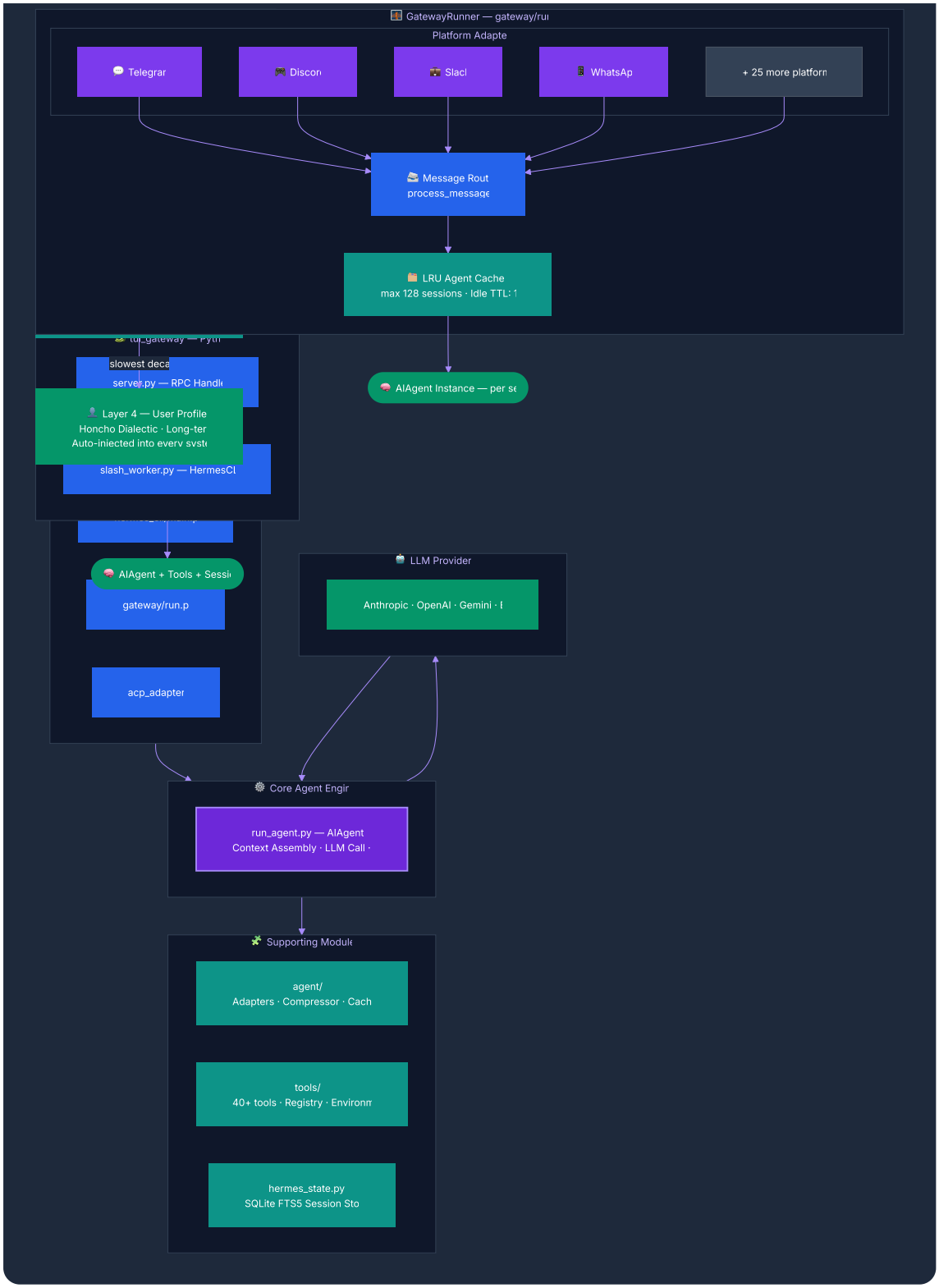
For Anthropic Claude models, Hermes supports **extended thinking** — a mode where the model explicitly reasons through a problem before answering. This is stored in the message history as "reasoning blocks" and gives you visibility into how the AI arrived at its answer.

```
# Example: Extended thinking stored in message history
{
  "role": "assistant",
  "reasoning": "Let me think through this step by step...",
  "content": "Based on my analysis, the answer is..."
}
```

Parallel Tool Execution

Hermes can run multiple tools simultaneously using the `delegate_task` tool, which spawns **sub-agents**. For example, when researching a topic, it might concurrently search five different web sources, read three files, and check a database — all at the same time, dramatically reducing wait times.

```
PARALLEL SUB-AGENT EXECUTION
```


GatewayRunner — gateway/rur
Platform Adapte
Telegrar
Discor
Slacl
WhatsAp
+ 25 more platforn
Message Rout
process_message
LRU Agent Cache
max 128 sessions · Idle TTL: 1
AIAgent Instance — per se
slowest deca
server.py — RPC Handle
Layer 4 — User Profile
Honcho Dialectic · Long-ter
Auto-injected into every syste
slash_worker.py — HermesCl
AIAgent + Tools + Sessi
gateway/run.p
acp_adapter
LLM Provider
Anthropic · OpenAI · Gemini · I
Core Agent Engir
run_agent.py — AIAgent
Context Assembly · LLM Call ·
Supporting Module
agent/
Adapters · Compressor · Cach
tools/
40+ tools · Registry · Environm
hermes_state.py
SQLite FTS5 Session Sto

SECTION 4

Architecture Deep Dive

> The complete system architecture — from Python core to TypeScript TUI to messaging gateway.

System Overview

```
HERMES AGENT — FULL SYSTEM ARCHITECTURE
```

The Six Major Components

1. Core Agent Engine — `run_agent.py`

The ~12,000-line `AIAgent` class is the heart of the system. It manages the entire conversation lifecycle: building context, calling the LLM, dispatching tool calls, handling retries, and streaming responses. Key sub-systems it orchestrates:

- **Prompt Builder** (`agent/prompt_builder.py`) — Assembles the system prompt dynamically based on enabled tools, active skills, memory context, and user preferences
- **Context Compressor** (`agent/context_compressor.py`) — Automatically summarises old messages when approaching the token limit
- **Prompt Caching** (`agent/prompt_caching.py`) — Caches the system prompt for Anthropic models to reduce API costs by up to 80%
- **Credential Pool** (`agent/credential_pool.py`) — Manages API keys across multiple providers, supporting rotation and fallback

2. Tool System — `tools/`

65 Python files implementing 40+ tools, each self-registering via a central registry pattern:

```
# How every tool registers itself (tools/web_tools.py)
from tools.registry import registry

@registry.register(
    name="web_search",
    description="Search the web for current information",
    parameters={"query": {"type": "string"}}
)
async def web_search(query: str) -> str:
    # Implementation...
    ...
```

3. Terminal Backends — `tools/environments/`

The terminal tool is backend-agnostic — the same tool code works across six different execution environments:

BACKEND	HOW IT WORKS	BEST FOR
Local	PTY (pseudo-terminal) directly on host OS	Development, personal use
Docker	Spins up isolated containers per session	Security, reproducibility
SSH	Remote execution on any SSH-accessible server	DevOps, remote management
Modal	Serverless functions, hibernates when idle	Cost-efficient cloud usage
Daytona	Persistent cloud dev environments	Team collaboration
Singularity	HPC-grade containers	Scientific computing

4. CLI Architecture — `cli.py` + `hermes_cli/`

The interactive terminal experience is built with:

- **Rich** — Beautiful terminal formatting: tables, panels, syntax-highlighted code, progress bars
- **prompt_toolkit** — Advanced input handling: multi-line editing, tab completion, keyboard shortcuts, command history
- **KawaiiSpinner** — Animated ASCII art displayed while waiting for AI responses

- **Skin Engine** — Data-driven theming: 4 built-in skins (default, ares, mono, slate), fully customisable

5. TUI Architecture — `ui-tui/` + `tui_gateway/`

The Terminal UI is a hybrid system: a TypeScript/React frontend talking to a Python backend over JSON-RPC:

```
TUI ARCHITECTURE
```

6. Messaging Gateway — `gateway/`

A single `GatewayRunner` manages up to 29 messaging platforms simultaneously. Each platform has a dedicated adapter translating platform-specific message formats to/from the common internal format. The gateway maintains an LRU cache of up to 128 active `AIAgent` instances — one per user/channel session — and evicts idle sessions after 1 hour.

Data Flow: From Telegram Message to AI Response

1. **Message Arrives via Telegram API**

   The Telegram adapter in `gateway/platforms/telegram/` receives a webhook event and parses the message text, sender ID, and chat ID.

2. **Session Lookup**

   The gateway looks up or creates an `AIAgent` instance for this sender, restoring their conversation history from SQLite.

3. **Agent Loop**

   The message is passed to the `AIAgent`, which runs the full conversation loop (context assembly → LLM → tools → response).

4. **Platform-Specific Formatting**

The response is formatted for Telegram's Markdown variant (different from Slack's or Discord's) and any length limits are handled.

5

Message Sent + Session Saved

The formatted response is sent via Telegram API, and the session state (including new messages) is persisted to SQLite for the next turn.

Configuration System

Hermes uses a layered configuration system. All settings live in `~/.hermes/config.yaml`. The interactive setup wizard (`hermes setup`) guides you through the initial configuration:

```yaml
# ~/.hermes/config.yaml — example configuration
model:
  default: "anthropic/claude-sonnet-4-6"
  auxiliary: "anthropic/claude-haiku-4-5"  # for quick tasks

tools:
  enabled:
    - web_search
    - terminal
    - read_file
    - write_file
    - browser_navigate
  terminal_backend: "local"  # or docker, ssh, modal...

memory:
  backend: "honcho"  # user profiling
  session_search: true

display:
  skin: "default"  # or ares, mono, slate
  streaming: true
```

SECTION 5

The Tool Ecosystem

> Every capability Hermes can exercise — the 40+ tools that give the agent its "hands".

Tool Categories

🌐 Web & Research Tools

TOOL	WHAT IT DOES	POWERED BY
`web_search`	Search the web and get structured results	Exa, Parallel search
`web_extract`	Extract full content from any URL	Firecrawl, httpx
`browser_navigate`	Control a real browser (click, type, scroll)	Browserbase
`browser_snapshot`	Take screenshot or read page accessibility tree	Browserbase

💻 Code & System Tools

TOOL	WHAT IT DOES
`terminal`	Execute shell commands in configured environment (local/Docker/SSH/Modal)
`execute_code`	Run Python/JS/bash code in an isolated sandbox with output capture
`process`	Start, monitor, and stop background processes

📁 File Management Tools

TOOL	WHAT IT DOES
`read_file`	Read any file (text, code, PDF, CSV) with smart chunking for large files
`write_file`	Create or overwrite files, with optional diff preview
`patch`	Apply targeted edits to existing files (like a surgical find-and-replace)
`search_files`	Search across files using regex or fuzzy matching

🧠 Memory & Knowledge Tools

TOOL	WHAT IT DOES
`memory`	Store and retrieve persistent facts, preferences, and notes
`session_search`	Full-text search across all past conversation sessions
`todo`	Agent-level task list that persists across the conversation

🎨 Creative & Media Tools

TOOL	WHAT IT DOES
`vision_analyze`	Analyse images, screenshots, or documents (using an auxiliary vision model)
`image_generate`	Generate images from text prompts using configured image generation APIs
`text_to_speech`	Convert text to audio (edge-tts for free, ElevenLabs for premium)

🔄 Orchestration & Automation Tools

TOOL	WHAT IT DOES
`delegate_task`	Spawn parallel sub-agents to handle independent sub-tasks concurrently
`cronjob`	Schedule tasks to run at specific times or intervals (cron expressions)
`send_message`	Send messages to users on any configured platform (Telegram, Slack, etc.)
`clarify`	Ask the user a clarifying question mid-task without breaking the flow

🏠 Home Automation Tools

A dedicated set of `ha_*` tools for Home Assistant integration: control lights, check sensors, run automations, and query device states — all from a conversational interface.

The Tool Registry Pattern

All tools follow the same self-registration pattern. When `run_agent.py` starts, it imports all files in `tools/` — each file calls `registry.register()` at import time. The result is a dynamic tool catalogue that can be extended simply by dropping a new Python file in the `tools/` directory.

> 🔌 **Extending Hermes with Custom Tools**
> Adding a new tool requires writing a single Python function decorated with `@registry.register()` . No changes needed to any other file. The tool automatically appears in the LLM's tool catalogue on the next restart.

Dangerous Command Detection

The `tools/approval.py` module contains a safeguard layer that detects potentially destructive terminal commands (e.g., `rm -rf` , `DROP TABLE` , `format`) and pauses execution to request human approval before proceeding. This is a critical safety mechanism for autonomous workflows.

SECTION 6

Memory & Learning System

> How Hermes builds a persistent, deepening understanding of you and your work across sessions.

The Memory Stack

Hermes has a multi-layered memory architecture. Each layer stores different types of information and operates at a different timescale:

```
MEMORY LAYERS — FASTEST TO SLOWEST DECAY
```

Memory Backend Options

The memory backend is a swappable plugin. Choose based on your needs:

BACKEND	TYPE	BEST FOR	SETUP
Honcho	Dialectic user modelling	Deep personal assistant use, learns your personality	API key (free tier)
Mem0	Vector DB (cloud)	Large memory stores, semantic search	API key
Holographic	Local graph DB	Fully private, relational memory	No API key needed
Hindsight	Short-term buffer	Session context, quick recall	No API key needed
RetainDB	Retention-based priority	Important facts surface more often over time	Minimal setup

The Session Search Engine

Every conversation Hermes has is stored in a SQLite database with **FTS5 full-text search** indexing. This means you can search across every conversation you've ever had:

```
# You can ask Hermes:
"What did I tell you about the Jenkins CI setup last month?"
# Hermes uses session_search to find the relevant conversation
# and recalls the exact context.
```

FTS5 supports **proximity searches** (words near each other), **phrase matching**, and **ranking by relevance** — not just simple keyword matching. The LLM then summarises the relevant excerpts in natural language.

The Self-Improving Skill Loop

The most distinctive feature of Hermes is its ability to **autonomously create and improve skills** from experience:

1. **Complex Task Detected**

   When Hermes completes a multi-step task that took many tool calls, it identifies this as a "trajectory" worth saving.

2. **Trajectory Saved**

The sequence of steps — tools used, decisions made, output format — is saved via `agent/trajectory.py`.

3

Skill Synthesised

Hermes writes a Python script that encodes the workflow as a reusable skill with parameters (the variable parts you'd change each time).

4

Skill Installed + Available

The skill is saved to `~/.hermes/skills/` and immediately available as a slash command. Future invocations are fast and consistent.

5

Skill Improves Over Time

Each time the skill is used, Hermes can refine it based on the outcome — correcting errors, adding edge case handling, and updating outputs.

🌱

Why This Matters

Most AI assistants get no better over time — they're stateless. Hermes compounds: every task it completes either adds to its memory or becomes a reusable skill. After months of use, a Hermes instance personalised to your workflow is dramatically more efficient than a fresh one.

SECTION 7

Multi-Platform Gateway

> How Hermes becomes available on 29+ messaging platforms simultaneously.

Supported Platforms

The gateway module allows a single Hermes instance to serve users across all major messaging platforms at once:

- 💬 Telegram
- 🎮 Discord
- 💼 Slack
- 📱 WhatsApp
- 📡 Signal
- 💬 WeChat
- 🐧 QQ
- 🔔 DingTalk
- 🪶 Feishu (Lark)
- 🏠 Home Assistant
- 📧 Email (SMTP)
- 🔵 Matrix
- 📟 IRC
- 📢 Zulip
- 📱 SMS (Twilio)
- + 14 more

How the Gateway Works

GATEWAY ARCHITECTURE

Session Persistence Across Platforms

Each user gets their own persistent session stored in SQLite. Importantly, **sessions are platform-aware but conversation-continuous** — your Hermes assistant remembers previous conversations regardless of whether you last spoke to it via Telegram or Slack.

The Cron Scheduler

Hermes can schedule tasks to run automatically using standard cron expressions. The scheduler (`cron/scheduler.py`) integrates with the gateway to deliver results to any configured platform:

```
# Examples of scheduled tasks
"0 9 * * 1-5"    → Run every weekday at 9am
"0 18 * * *"     → Run every day at 6pm
"*/30 * * * *"  → Run every 30 minutes

# Usage from chat:
/cron set "daily at 8am, check my emails and send me a summary"
/cron set "every Monday at 9am, pull our sales data and make a report"
```

Natural Language Scheduling

You don't need to know cron syntax. Tell Hermes "remind me every Friday afternoon to send the team update" and it converts it to the correct cron expression automatically.

ACP Integration — IDE Support

The `acp_adapter/` module implements the **Agent Client Protocol**, allowing Hermes to be used directly inside VS Code, Zed, and JetBrains IDEs. The adapter runs as a background server that editors communicate with via JSON-RPC — you get Hermes' full tool ecosystem (web search, file management, code execution) inside your editor without switching to the terminal.

SECTION 8

Business Use Cases

> Real-world applications of Hermes Agent for organisations of all sizes.

🏢

Why Businesses Choose Self-Hosted AI

Commercial AI tools send your data to third-party servers. For businesses handling sensitive customer data, financial records, or proprietary IP, self-hosted AI like Hermes provides the intelligence of frontier models with the data control of on-premises software.

📊

Business Intelligence & Market Research

Automating competitive intelligence gathering and market analysis

Hermes can continuously monitor competitors, industry news, and market signals — then synthesise findings into structured reports delivered to your team on a schedule.

Competitor Monitoring

Schedule daily checks of competitors' websites, job postings, press releases, and social media for strategic signals

Market Reports

Auto-generate weekly market summaries by pulling data from multiple sources, running analysis, and formatting for stakeholders

Pricing Intelligence

Monitor competitor pricing changes and product launches, alerting the sales team in Slack when key changes occur

Research Synthesis

Summarise long technical papers, industry reports, or regulatory updates into decision-ready briefs

Customer Support Automation

An always-on support agent that escalates intelligently

Deploy Hermes as a first-line support agent on Telegram, Slack, or your website. It handles common queries, looks up order details, and escalates complex issues to human agents with full context.

FAQ Handling

Train Hermes on your documentation and have it answer product questions 24/7 across all your support channels

Order Lookup

Connect to your database via custom tool — customers ask about order status and Hermes queries it in real-time

Ticket Triage

Classify incoming support tickets, assign priority, and route to the right team — all automatically

Multi-lingual Support

Handle support queries in any language the underlying LLM supports, responding in the customer's language

DevOps & Engineering Automation

AI-assisted development, operations, and incident response

Engineering teams can deploy Hermes as a DevOps assistant that monitors systems, diagnoses issues, drafts PR descriptions, reviews code, and can take corrective actions when configured to do so.

Code Review

Automatically review PRs for security issues, code quality, and style — posting comments directly on GitHub

Incident Response

When an alert fires, Hermes analyses logs, checks related services, and posts a diagnostic summary to the ops Slack channel

Documentation Generation

Read new code commits and auto-update documentation, changelogs, and API references

Deployment Automation

Manage CI/CD pipeline steps via terminal commands, handling common deployment failures with predefined runbooks

Operations & Workflow Automation

Eliminating repetitive operational tasks across departments

Report Generation

Pull data from databases, generate formatted reports (PDF, Excel, slides), and distribute to stakeholders on schedule

Meeting Intelligence

Process meeting recordings/transcripts: extract action items, decisions, and deadlines; distribute follow-up summaries

Contract Review

Analyse contracts for key clauses, risks, and deadlines — surface important items for legal team review

Data Pipeline

Schedule data processing jobs, transform formats, validate data quality, and alert on anomalies

Compliance & Security Monitoring

Automated risk detection and regulatory compliance checking

Policy Compliance

Regularly scan systems against security policies and generate compliance reports for auditors

Vulnerability Scanning

Run security scans via terminal tools, summarise findings in plain English, and create prioritised remediation tickets

GDPR/Data Audits

Scan codebases and databases for PII handling issues; generate data inventory reports

Regulatory Updates

Monitor regulatory sources for changes affecting your industry; summarise and alert legal/compliance teams

Deployment Model for Businesses

Recommended Enterprise Setup

- ✓ Deploy Hermes on a private VPS or internal server using Docker
- ✓ Configure AWS Bedrock as the LLM provider for maximum data sovereignty
- ✓ Enable Slack and email gateways for company-wide access

- ✓ Store all sessions in an encrypted SQLite database on internal storage
- ✓ Use Docker environment backend to isolate code execution
- ✓ Set up scheduled skills for daily/weekly reporting workflows
- ✓ Configure per-user memory profiles for personalised assistance

💰

Cost Consideration

Hermes itself is free. You pay only for the LLM API calls you make. With prompt caching enabled (Anthropic models), system prompt costs are reduced by up to 80%. For high-volume use, local/open-source models (Ollama + Llama) bring API costs to zero.

SECTION 9

Content Creator Use Cases

> How Hermes accelerates content workflows for YouTubers, podcasters, bloggers, and social media creators.

> *"The biggest bottleneck for content creators isn't creativity — it's the operational overhead: research, writing, editing, scheduling, and analytics. Hermes can handle the entire operational layer, freeing you to focus on the creative work only you can do."*

YouTube & Video Content

From idea to upload — automating the full video content pipeline

Content Research

Give Hermes a topic — it searches YouTube trends, reads top-ranking articles, and synthesises a comprehensive research brief with key talking points

Script Writing

Based on the research brief, generate a full video script in your tone and style (it learns your style from past scripts you share)

Thumbnail Concepts

Generate image prompts for thumbnail designs and create draft thumbnails via image generation tools

Metadata Optimisation

Write SEO-optimised titles, descriptions, and tag lists for each video based on keyword research

Transcript Repurposing

Turn video transcripts into blog posts, Twitter threads, LinkedIn articles, and email newsletters automatically

Analytics Summary

Scheduled weekly report: pull YouTube analytics data and summarise performance trends with recommendations

Podcasting

Automate the production and distribution workflow

Guest Research

Given a guest's name and LinkedIn, compile a comprehensive briefing: background, notable work, talking points, suggested questions

Show Notes

From a transcript or recording, generate structured show notes with timestamps, key quotes, and resource links

Episode Summaries

Write different length summaries (tweet, paragraph, full description) for different distribution channels

Clip Identification

Analyse transcripts to identify the best 60-second clips for social media promotion

Blogging & Written Content

Research, writing, and SEO in a single workflow

Topic Ideation

Research trending topics in your niche, identify content gaps versus competitors, and suggest a 3-month content calendar

SEO Research

Find the best keywords for a topic, analyse top-ranking posts for structure and coverage, create an SEO-informed outline

First Draft

Generate a detailed first draft from your outline — in your voice and style, with proper sourcing and internal link suggestions

Content Refreshing

Identify older posts that need updating, research what's changed, and produce an updated version with new stats and sources

Social Media Management

A complete social media presence — researched, written, and scheduled

Content Repurposing

Turn a single YouTube video or blog post into a week's worth of social content: Twitter threads, Instagram carousels, LinkedIn posts, TikTok scripts

Trend Monitoring

Daily check of trending topics in your niche — surface opportunities to post timely, relevant content before the trend peaks

Engagement Responses

Draft responses to common comment types in your voice — you review and approve, Hermes handles the writing

Analytics Digest

Weekly summary of best-performing content across platforms with actionable insights about what to create more of

The Content Creator Workflow — End to End

Here's how a creator might use Hermes to produce a single piece of content from zero to published:

1. **Idea + Research (15 min → 2 min)**

   Type: "Research 'beginner photography tips' for a YouTube video." Hermes searches YouTube, reads top articles, checks Google Trends, and returns a structured brief.

2. **Script Draft (4 hours → 20 min)**

   Type: "Write a 10-minute YouTube script on this topic in my conversational style." Hermes references past scripts you've shared and writes a full draft.

3. **You Film & Edit (Your Creative Work)**

   The part only you can do — the actual filming, performance, and editing.

4. **Metadata + Thumbnail (2 hours → 10 min)**

   Share the transcript: Hermes generates SEO title options, a description, tags, and thumbnail concepts to choose from.

5. **Distribution (1 hour → 5 min)**

   Hermes repurposes the transcript into a Twitter thread, LinkedIn post, Instagram caption, and email newsletter — all in your voice.

> ⏱️ **Time Savings in Practice**
>
> A typical content creator spends 6–8 hours on research, writing, and distribution per piece. With Hermes handling the operational work, this can drop to under 1 hour — meaning you can produce 6–8× more content, or reclaim those hours for higher-leverage creative work.

Personalisation: Teaching Hermes Your Voice

The memory system means Hermes learns your style over time. You can accelerate this by:

- **Sharing past work** — Give Hermes your best blog posts or scripts and say "this is my writing style — remember it."
- **Giving feedback** — When it writes something off-brand, correct it. The correction is stored in memory and applied to all future drafts.
- **Creating style skills** — You can save a "write in my voice" skill with all your style notes encoded, invocable with a single slash command.

SECTION 10

Developer Guide & Getting Started

> From installation to building custom tools and skills.

Installation

Quick Install (Recommended)

```
# One-line install script
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scrip

# After install:
source ~/.bashrc  # or ~/.zshrc
hermes setup      # interactive setup wizard
hermes            # start chatting!
```

Development Install (Full Control)

```
git clone --recurse-submodules https://github.com/NousResearch/hermes-agent.git
cd hermes-agent

# Using uv (recommended Python package manager)
uv venv venv --python 3.11
source venv/bin/activate
uv pip install -e ".[all,dev]"

# OR using pip
python -m venv venv
source venv/bin/activate
pip install -e ".[all,dev]"

# Run initial setup
hermes setup
```

Core Commands

COMMAND	WHAT IT DOES
`hermes`	Start interactive CLI chat session
`hermes --tui`	Launch the Terminal UI (React/Ink)
`hermes setup`	Run interactive configuration wizard
`hermes model`	Switch AI model/provider
`hermes tools`	Enable/disable tools
`hermes skills`	Browse, install, and manage skills
`hermes gateway`	Configure and start messaging gateway
`hermes cron list`	View scheduled tasks
`hermes status`	System health check
`hermes doctor`	Diagnose configuration issues

Useful Slash Commands (In Chat)

COMMAND	WHAT IT DOES
`/model provider:model`	Switch model mid-conversation
`/skills`	Browse and install skills
`/retry`	Retry the last message
`/compress`	Manually compress conversation context
`/clear`	Start a fresh conversation
`/`	Run an installed skill

Building a Custom Tool

Creating a new tool for Hermes requires a single Python file in the `tools/` directory:

```python
# tools/my_custom_tool.py
from tools.registry import registry
import httpx

@registry.register(
    name="get_stock_price",
    description="Get the current stock price for a ticker symbol",
    parameters={
        "ticker": {
            "type": "string",
            "description": "Stock ticker symbol, e.g. AAPL"
        }
    }
)
async def get_stock_price(ticker: str) -> str:
    async with httpx.AsyncClient() as client:
        resp = await client.get(
            f"https://api.example.com/price/{ticker}"
        )
    data = resp.json()
    return f"${data['price']} USD ({ticker})"
```

That's it. On next run, the tool appears in the LLM's tool catalogue automatically.

Building a Custom Skill

Skills live in `~/.hermes/skills/<skill-name>/` and consist of:

```
# ~/.hermes/skills/weekly-report/SKILL.md
---
name: weekly-report
description: Generate a weekly business performance report
trigger: /weekly-report
---
Pull the last 7 days of sales data, compare to the previous week,
highlight top performers, and format as a Slack-ready summary.

# ~/.hermes/skills/weekly-report/main.py
import sys

args = sys.argv[1:]
week = args[0] if args else "current"

print(f"Generating weekly report for: {week}")
print("Focus on: revenue, top SKUs, and week-over-week delta.")
```

Key Files Reference

FILE / DIRECTORY	PURPOSE
`run_agent.py`	Core AIAgent class (12K lines) — main conversation engine
`cli.py`	Interactive CLI — Rich + prompt_toolkit (10.8K lines)
`hermes_cli/main.py`	All `hermes` CLI subcommands
`tools/registry.py`	Tool registration and dispatch system
`hermes_state.py`	SQLite session store with FTS5 search
`gateway/run.py`	Multi-platform messaging gateway
`agent/prompt_builder.py`	Dynamic system prompt assembly
`agent/context_compressor.py`	Automatic context window management
`cron/scheduler.py`	Cron job scheduler
`skills/`	Built-in skills (27 categories)
`plugins/memory/`	Memory backend plugins (8 options)
`~/.hermes/config.yaml`	Your personal configuration

Technology Stack Summary

🐍 Python Core

`openai`, `anthropic`, `rich`, `prompt_toolkit`, `pydantic`, `httpx`, `sqlite3`, `asyncio`

⚛ TypeScript TUI

React 19, Ink (terminal renderer), Vite, JSON-RPC protocol, TypeScript

🗄 Storage

SQLite with FTS5 for sessions, YAML for config, optional vector DBs for semantic memory

📦 Packaging

`uv` for Python deps, `npm` for Node, Docker for containerisation, Nix for reproducibility